

Лабораторна робота №3

Тема: Умовні переходи

Мета: Опанувати навички побудови розгалужених алгоритмів у мові Python; навчитися використовувати логічні операції, булеву алгебру та умовні конструкції (if, elif, else, а також оператор множинного вибору match-case) для розв'язання прикладних і логічних задач.

КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

1. Логічний тип даних та булева алгебра

У Python логічний тип даних (bool) має лише два значення: True (істина) та False (хибність). Результатом порівняння величин за допомогою операторів відношення (>, <, ==, !=, >=, <=) завжди є булеве значення.

Для побудови складних умов використовуються логічні операції:

- **and (логічне І):** повертає True, якщо **обидві** умови істинні.
- **or (логічне АБО):** повертає True, якщо **хоча б одна** умова істинна.
- **not (логічне НЕ):** інвертує значення умови (змінює True на False і навпаки).

2. Конструкція розгалуження if-elif-else

Для керування потоком виконання програми залежно від виконання умов використовують умовний оператор:

Синтаксис конструкції

```
if умова_1:
    # блок коду, якщо умова_1 істинна
elif умова_2:
    # блок коду, якщо умова_2 істинна
else:
    # блок коду, якщо всі попередні умови хибні
```

3. Множинний вибір (match-case)

Починаючи з Python 3.10, для організації множинного вибору замість класичного switch використовується сучасна конструкція match-case:

Синтаксис конструкції

match предмет_перевірки:

```
case шаблон_1:
    # блок коду для першого шаблону
case шаблон_2 if умова:
    # блок коду з додатковою умовою (стражем)
case шаблон_3 | шаблон_4:
    # блок коду для кількох альтернативних шаблонів
```

case _:

блок коду за замовчуванням (якщо жоден шаблон не підійшов)

Приклад. Написати програму-інтерпретатор команд. Користувач вводить текстову команду (наприклад, "help", "status", "exit" або командний код з числовим параметром). За допомогою оператора match-case програма повинна розпізнати команду та вивести відповідне повідомлення або виконати відповідну дію.

Приклад програмного коду

Python

```
print("--- Симулятор командного термінала ---")
print("Доступні команди: help, status, reload, exit")

# Отримуємо команду від користувача та приводимо до
нижнього регістру
command = input("Введіть команду: ").strip().lower()

# Використовуємо оператор множинного вибору match-case
match command:
    case "help":
        print("Довідка: Ви можете використовувати
команди help, status, reload або exit.")

    case "status":
        print("Система працює стабільно. Помилки не
виявлено.")

    case "reload" | "restart": # Об'єднання кількох
варіантів через логічне АБО (|)
        print("Перезавантаження системи... Будь ласка,
зачекайте.")

    case "exit" | "quit":
        print("Вихід із програми. До побачення!")

    case unknown_cmd: # Шаблон за замовчуванням
(catch-all), аналог 'else' або 'default'
        print(f"Помилка: Команду '{unknown_cmd}' не
розпізнано. Введіть 'help' для довідки.")
```

Пояснення роботи match-case у цьому прикладі:

1. **match command:** Змінна `command` передається на перевірку в блок `match`. Python послідовно порівнюватиме її значення з варіантами у гілках `case`.

2. **Точні збіги (case "help":, case "status":):** Якщо введена команда дорівнює конкретному рядку, виконується відповідний блок коду.

3. **Об'єднання варіантів через вертикальну риску (case "exit" | "quit":):** Конструкція | дозволяє опрацьовувати кілька різних варіантів введення (наприклад, користувач може ввести як "exit", так і "quit"), виконуючи для них однакову дію.

4. **Змінна-шаблон (case unknown_cmd:):** Якщо жоден із попередніх шаблонів не підійшов, спрацьовує останній рядок. При цьому значення змінної записується у змінну unknown_cmd (що дозволяє динамічно використати його всередині блоку, наприклад, вивести текст помилки разом із введеним словом). Це безпечний аналог блока default у класичних мовах програмування.

ЗАВДАННЯ ДЛЯ САМОСТІЙНОЇ РОБОТИ

Виконайте завдання, розбиті на два рівні складності. Кожне завдання передбачає розробку повноцінної програми на Python з обробкою вхідних даних та виведенням результату.

Рівень I (Базовий / Середній складність)

1. **Завдання 1.1 («Розрахунок заробітної плати з урахуванням понаднормових»):** На станції ремонту автомобілів працівник, який працює понад 40 годин на тиждень, отримує в 1,5 рази більше від звичайної погодинної оплати за кожну понаднормову годину. Програма отримує на вхід два цілих числа — кількість робочих годин і ставку погодинної оплати.

- *Приклад вхідних даних:* 45 годин, ставка 15.
- *Приклад вихідних даних:* 712.50.

2. **Завдання 1.2 («Перевірка співвідношення кубів та суми цифр трицифрового числа»):** Дано трицифрове число. Визначити, чи рівний квадрат суми цифр числа сумі кубів його цифр. Програма повинна виводити логічне значення (True або False).

- *Приклад вхідних даних:* 123, 210, 150.
- *Приклад вихідних даних:* True, True, False.

3. **Завдання 1.3 («Визначення типу дня року»):** Дано ціле число k ($1 \leq k \leq 365$). Визначити, яким буде k -й день року: вихідним (субота чи неділя) або робочим, якщо 1 січня — понеділок (вважаємо рік не високосним).

- *Приклад вхідних даних:* 5, 7, 56.
- *Приклад вихідних даних:* working day, day off, day off.

Рівень II. Підвищена складність – логічні та алгоритмічні задачі.

1. **Завдання 2.1 «Пошук унікального елемента з чотирьох чисел».** Відомо, що з чотирьох цілих чисел одне відмінне від трьох інших, рівних між собою. Розробити програму, яка визначає порядковий номер цього унікального числа.

2. **Завдання 2.2 «Банкомат – мінімальна кількість купюр.** Банкомат містить у достатній кількості купюри номіналом 10, 20, 50, 100, 200 і 500 гривень. Знайти найменшу кількість купюр, якою можна видати суму в n гривень, або вивести -1, якщо вказану суму видати не можна.

- *Вхідні дані:* Одне число n ($1 \leq n \leq 1000000$).
- *Вихідні дані:* Найменша кількість купюр або -1.

3. **Завдання 2.3 «Парні числа без умовних операторів».** Дано чотири цілих числа. Визначити, скільки з них є парними. Завдання передбачає розробку оптимізованого способу розв'язання **без використання оператора розгалуження (if)** (наприклад, за допомогою математичних перетворень або булевої арифметики).